

Name: Kinyua Seaman

Date: June 4th, 2026

JKUAT

College/School: COPAS/SCIT

Semeter: Feb-May 2026

Degree: M. Sc AI

Unit Code: ICS 3308

Unit Name: Deep Learning

QUESTION ONE

Part A: Perceptron Learning Rule

Problem Statement: Design and implement a Perceptron that is able to map all the inputs to their correct target. Initialize all weights and bias to 0 and a learning rate of 1.

Mathematical Formulation: Given the input vector $X = [x_1, x_2, x_3, x_4]^T$, target $t \in \{-1, 1\}$, weights $W = [w_1, w_2, w_3, w_4]$, and bias b . The net input is calculated as:

$$y_{in} = b + \sum_{i=1}^4 x_i w_i = b + W \cdot X$$

The activation function (with threshold $\theta = 0$) is:

$$y = f(y_{in}) = \begin{cases} 1 & \text{if } y_{in} > 0 \\ 0 & \text{if } y_{in} = 0 \\ -1 & \text{if } y_{in} < 0 \end{cases}$$

If the output y does not match target t , the weights and bias are updated using:

$$W^{(new)} = W^{(old)} + \alpha \cdot t \cdot X$$

$$b^{(new)} = b^{(old)} + \alpha \cdot t$$

where $\alpha = 1$ is the learning rate. If $y = t$, no updates are made.

```
In [1]: import numpy as np
import pandas as pd
pd.set_option('display.max_columns', None)
pd.set_option('display.width', 1000)

# Define input patterns and target labels
X = np.array([
```

```

    [1, 1, 1, 1],
    [-1, 1, -1, -1],
    [1, 1, 1, -1],
    [1, -1, -1, 1]
])
targets = np.array([1, 1, -1, -1])

# Initial parameters
W = np.zeros(4, dtype=float)
b = 0.0
lr = 1.0

# Perceptron step activation function
def step_activation(yin):
    if yin > 0:
        return 1
    elif yin < 0:
        return -1
    else:
        return 0

# Training simulation
history = []
converged = False
epoch = 0
max_epochs = 10

while not converged and epoch < max_epochs:
    epoch += 1
    epoch_changed = False

    for i in range(len(X)):
        x_i = X[i]
        t = targets[i]
        yin = np.dot(x_i, W) + b
        y = step_activation(yin)
        e = t - y

        # Capture state before weights update
        old_W = W.copy()
        old_b = b

        # If actual does not match target, perform update
        if y != t:
            W += lr * t * x_i
            b += lr * t
            epoch_changed = True

    history.append({
        "Epoch": epoch,
        "X": str(x_i.tolist()),
        "t": t,
        "yin": yin,
        "y": y,
        "e": e,
        "W": str(W.tolist()),

```

```

        "b": b
    })

    if not epoch_changed:
        converged = True

df_history = pd.DataFrame(history)
df_history

```

Epoch	X	t	yin	y	e	W	b
1	[1, 1, 1, 1]	1	0.0	0	1	[1.0, 1.0, 1.0, 1.0]	1.0
1	[-1, 1, -1, -1]	1	-1.0	-1	2	[0.0, 2.0, 0.0, 0.0]	2.0
1	[1, 1, 1, -1]	-1	4.0	1	-2	[-1.0, 1.0, -1.0, 1.0]	1.0
1	[1, -1, -1, 1]	-1	1.0	1	-2	[-2.0, 2.0, 0.0, 0.0]	0.0
2	[1, 1, 1, 1]	1	0.0	0	1	[-1.0, 3.0, 1.0, 1.0]	1.0
2	[-1, 1, -1, -1]	1	3.0	1	0	[-1.0, 3.0, 1.0, 1.0]	1.0
2	[1, 1, 1, -1]	-1	3.0	1	-2	[-2.0, 2.0, 0.0, 2.0]	0.0
2	[1, -1, -1, 1]	-1	-2.0	-1	0	[-2.0, 2.0, 0.0, 2.0]	0.0
3	[1, 1, 1, 1]	1	2.0	1	0	[-2.0, 2.0, 0.0, 2.0]	0.0
3	[-1, 1, -1, -1]	1	2.0	1	0	[-2.0, 2.0, 0.0, 2.0]	0.0
3	[1, 1, 1, -1]	-1	-2.0	-1	0	[-2.0, 2.0, 0.0, 2.0]	0.0
3	[1, -1, -1, 1]	-1	-2.0	-1	0	[-2.0, 2.0, 0.0, 2.0]	0.0

Part B: Hopfield Network

Problem Statement: Train a Hopfield network to recall the stored patterns;

$P_1 = [1, -1, 1, -1]^T$ and $P_2 = [-1, 1, -1, 1]^T$. Show how it behaves given a noisy pattern $P = [1, 0, 1, -1]^T$.

Mathematical Formulations:

1. **Weight Matrix (W) Calculation:** Standard Hopfield networks use Hebbian learning with zero diagonal (to prevent self-connections):

$$W_{ij} = \sum_{\mu=1}^M P_i^{\mu} P_j^{\mu} \quad \text{for } i \neq j, \quad W_{ii} = 0$$

Here we have $M = 2$ stored patterns of size $N = 4$:

$$W = P_1 P_1^T + P_2 P_2^T - 2I$$

Since $P_2 = -P_1$, we have $P_1 P_1^T = P_2 P_2^T$, so:

$$W = 2 \cdot P_1 P_1^T - 2I = \begin{pmatrix} 0 & -2 & 2 & -2 \\ -2 & 0 & -2 & 2 \\ 2 & -2 & 0 & -2 \\ -2 & 2 & -2 & 0 \end{pmatrix}$$

2. **Recall Update Rule:** The activation function is:

$$x_i^{(new)} = \text{sgn} \left(\sum_{j=1}^N W_{ij} x_j \right) = \begin{cases} 1 & \text{if } \text{net}_i > 0 \\ -1 & \text{if } \text{net}_i < 0 \\ x_i^{(old)} & \text{if } \text{net}_i = 0 \end{cases}$$

```
In [1]: # Hopfield Weight Matrix calculation
P1 = np.array([1, -1, 1, -1])
P2 = np.array([-1, 1, -1, 1])

# Outer product sum
W_hopfield = np.outer(P1, P1) + np.outer(P2, P2)
np.fill_diagonal(W_hopfield, 0)

print("Hopfield Network Weight Matrix W:")
print(W_hopfield)

# Noisy Input Pattern
P_noisy = np.array([1, 0, 1, -1])

# Synchronous Recall
net_input = W_hopfield @ P_noisy
P_recalled_sync = np.where(net_input > 0, 1, np.where(net_input < 0, -1, P_noisy))

print("\n--- Synchronous Recall ---")
print(f"Noisy Pattern P:      {P_noisy}")
print(f"Net Input W @ P:      {net_input}")
print(f"Recalled Pattern:      {P_recalled_sync}")
print(f"Matches Stored P1?    {np.array_equal(P_recalled_sync, P1)}")

# Asynchronous Recall (node by node)
print("\n--- Asynchronous Recall (node-by-node update) ---")
state = P_noisy.copy().astype(float)
for idx in range(len(state)):
    net_val = np.dot(W_hopfield[idx], state)
    old_val = state[idx]
    if net_val > 0:
        state[idx] = 1
    elif net_val < 0:
        state[idx] = -1
    print(f"Step {idx+1}: Update node {idx+1} | Net input = {net_val:+.1f} | State tr
print(f"Final Asynchronous Recalled State: {state.astype(int)}")
```

Hopfield Network Weight Matrix W:

```
[[ 0 -2  2 -2]
 [-2  0 -2  2]
 [ 2 -2  0 -2]
 [-2  2 -2  0]]
```

--- Synchronous Recall ---

```
Noisy Pattern P:      [ 1  0  1 -1]
Net Input W @ P:      [ 4 -6  4 -4]
Recalled Pattern:      [ 1 -1  1 -1]
Matches Stored P1?    True
```

--- Asynchronous Recall (node-by-node update) ---

```
Step 1: Update node 1 | Net input = +4.0 | State transition: 1.0 -> 1.0
Step 2: Update node 2 | Net input = -6.0 | State transition: 0.0 -> -1.0
Step 3: Update node 3 | Net input = +6.0 | State transition: 1.0 -> 1.0
Step 4: Update node 4 | Net input = -6.0 | State transition: -1.0 -> -1.0
Final Asynchronous Recalled State: [ 1 -1  1 -1]
```

QUESTION TWO

Part A: MLP Forward Pass Implementation (PyTorch)

Problem Statement: Using PyTorch-style tensor operations such as `@matmul` or `@add` provide an implementation that implements the forward pass of a 2-layer Multi-Layer Perceptron (MLP) for a single input vector X .

Architecture:

- Input vector X of size d_{in} .
- Layer 1 (Hidden Layer): weight matrix W_1 of shape (d_{hidden}, d_{in}) , bias b_1 of shape $(d_{hidden},)$. Uses ReLU activation.
- Layer 2 (Output Layer): weight matrix W_2 of shape (d_{out}, d_{hidden}) , bias b_2 of shape $(d_{out},)$.
- Mathematical pass:

$$H = \text{ReLU}(W_1 X + b_1)$$

$$Y = W_2 H + b_2$$

In [1]: `import torch`

```
class TwoLayerMLP(torch.nn.Module):
    def __init__(self, input_dim, hidden_dim, output_dim):
        super(TwoLayerMLP, self).__init__()
        # Initialize weights and biases manually to show clear tensor mapping
        self.W1 = torch.nn.Parameter(torch.randn(hidden_dim, input_dim))
        self.b1 = torch.nn.Parameter(torch.zeros(hidden_dim))
        self.W2 = torch.nn.Parameter(torch.randn(output_dim, hidden_dim))
        self.b2 = torch.nn.Parameter(torch.zeros(output_dim))

    def forward(self, X):
```

```

    # Ensure single input vector X has shape (input_dim,)
    # Layer 1 forward: H = ReLU( W1 @ X + b1 )
    # Use torch.add and torch.matmul as requested
    h_net = torch.add(torch.matmul(self.W1, X), self.b1)
    h_act = torch.relu(h_net)

    # Layer 2 forward: Y = W2 @ H + b2
    y_net = torch.add(torch.matmul(self.W2, h_act), self.b2)
    return y_net

# Instantiation and test
input_size, hidden_size, output_size = 8, 16, 4
mlp = TwoLayerMLP(input_size, hidden_size, output_size)

# Single input vector X of shape (input_dim,)
X_single = torch.randn(input_size)
output = mlp(X_single)
print(f"Input Shape: {X_single.shape}")
print(f"Output Shape: {output.shape}")
print(f"Output Tensor:\n{output}")

```

```

Input Shape: torch.Size([8])
Output Shape: torch.Size([4])
Output Tensor:
tensor([ 1.0426, -0.0280,  1.2920, -0.7456], grad_fn=<AddBackward0>)

```

Part B: TensorFlow Computer Vision Data Pipeline (MNIST)

Problem Statement: Create a TensorFlow pipeline for MNIST dataset that performs Min-Max scaling within a tf.data pipeline. Ensure the data is shuffled, batched, and prefetched before being passed into a custom training loop.

Pipeline Stages:

1. **Load:** Load the raw MNIST dataset.
2. **Transform:** Scale pixel values from `[0, 255]` to `[0.0, 1.0]` using Min-Max scaling.
3. **Shuffle:** Shuffle the dataset with a buffer size.
4. **Batch:** Batch the dataset.
5. **Prefetch:** Overlap the preprocessing and model execution.
6. **Train:** Iterate over the dataset in a custom training loop.

```

In [1]: import tensorflow as tf

# 1. Load MNIST dataset
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()
y_train = y_train.astype('int64')

# Create tf.data Dataset from slices
mnist_dataset = tf.data.Dataset.from_tensor_slices((x_train, y_train))

# 2. Min-Max Scaling function
def scale_min_max(image, label):
    # Scale image pixels to range [0.0, 1.0] from [0, 255]

```

```

        image = tf.cast(image, tf.float32) / 255.0
        return image, label

# 3. Apply transformation, shuffle, batch, and prefetch
buffer_size = 10000
batch_size = 64

processed_dataset = (
    mnist_dataset
    .map(scale_min_max, num_parallel_calls=tf.data.AUTOTUNE)
    .shuffle(buffer_size)
    .batch(batch_size)
    .prefetch(tf.data.AUTOTUNE)
)

# Mock Model & Loss/Optimizer for Custom Training Loop demonstration
model = tf.keras.Sequential([
    tf.keras.Input(shape=(28, 28)),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dense(10)
])
loss_fn = tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True)
optimizer = tf.keras.optimizers.Adam()

# Custom Training Loop Demonstration (1 epoch, first 3 batches)
print("Executing Custom Training Loop...")
for epoch in range(1):
    print(f"Epoch {epoch + 1}")
    for batch_idx, (images, labels) in enumerate(processed_dataset.take(3)):
        with tf.GradientTape() as tape:
            logits = model(images, training=True)
            loss_value = loss_fn(labels, logits)

            grads = tape.gradient(loss_value, model.trainable_variables)
            optimizer.apply_gradients(zip(grads, model.trainable_variables))

        print(f"  Batch {batch_idx + 1} | Loss: {loss_value.numpy():.4f}")

```

Executing Custom Training Loop...

Epoch 1

Batch 1 | Loss: 2.3026

Batch 2 | Loss: 2.2912

Batch 3 | Loss: 2.2801

QUESTION THREE

Part A: 2D Convolution Operation

Problem Statement: Convolution is a key operation in CNN. Use the input image and the filter given to show the resulting feature map.

Input Image (I):

$$\begin{pmatrix} 1 & 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 & 0 \\ 0 & 1 & 1 & 0 & 0 \end{pmatrix}$$

Filter (F):

$$\begin{pmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 1 \end{pmatrix}$$

Convolution Math Calculation Walkthrough: Let the output feature map be O of shape 3×3 .

$$O(r, c) = \sum_{i=0}^2 \sum_{j=0}^2 I(r + i, c + j) \cdot F(i, j)$$

Because the filter elements are:

- $F(0, 0) = 1, F(0, 1) = 0, F(0, 2) = 1$
- $F(1, 0) = 0, F(1, 1) = 1, F(1, 2) = 0$
- $F(2, 0) = 1, F(2, 1) = 0, F(2, 2) = 1$

The formula simplifies to:

$$O(r, c) = I(r, c) + I(r, c + 2) + I(r + 1, c + 1) + I(r + 2, c) + I(r + 2, c + 2)$$

Let's compute each element:

1. $O(0, 0) = I(0, 0) + I(0, 2) + I(1, 1) + I(2, 0) + I(2, 2) = 1 + 1 + 1 + 0 + 1 = 4$
2. $O(0, 1) = I(0, 1) + I(0, 3) + I(1, 2) + I(2, 1) + I(2, 3) = 1 + 0 + 1 + 0 + 1 = 3$
3. $O(0, 2) = I(0, 2) + I(0, 4) + I(1, 3) + I(2, 2) + I(2, 4) = 1 + 0 + 1 + 1 + 1 = 4$
4. $O(1, 0) = I(1, 0) + I(1, 2) + I(2, 1) + I(3, 0) + I(3, 2) = 0 + 1 + 0 + 0 + 1 = 2$
5. $O(1, 1) = I(1, 1) + I(1, 3) + I(2, 2) + I(3, 1) + I(3, 3) = 1 + 1 + 1 + 0 + 1 = 4$
6. $O(1, 2) = I(1, 2) + I(1, 4) + I(2, 3) + I(3, 2) + I(3, 4) = 1 + 0 + 1 + 1 + 0 = 3$
7. $O(2, 0) = I(2, 0) + I(2, 2) + I(3, 1) + I(4, 0) + I(4, 2) = 0 + 1 + 0 + 0 + 1 = 2$
8. $O(2, 1) = I(2, 1) + I(2, 3) + I(3, 2) + I(4, 1) + I(4, 3) = 0 + 1 + 1 + 1 + 0 = 3$
9. $O(2, 2) = I(2, 2) + I(2, 4) + I(3, 3) + I(4, 2) + I(4, 4) = 1 + 1 + 1 + 1 + 0 = 4$

Thus, the final feature map is:

$$\begin{pmatrix} 4 & 3 & 4 \\ 2 & 4 & 3 \\ 2 & 3 & 4 \end{pmatrix}$$

```
In [1]: # NumPy Verification of 2D Convolution
I = np.array([
    [1, 1, 1, 0, 0],
    [0, 1, 1, 1, 0],
    [0, 0, 1, 1, 1],
    [0, 0, 1, 1, 0],
    [0, 1, 1, 0, 0]
])

F = np.array([
    [1, 0, 1],
    [0, 1, 0],
    [1, 0, 1]
])

# Perform 2D convolution
out_r = I.shape[0] - F.shape[0] + 1
out_c = I.shape[1] - F.shape[1] + 1
O = np.zeros((out_r, out_c), dtype=int)

for r in range(out_r):
    for c in range(out_c):
        patch = I[r:r+F.shape[0], c:c+F.shape[1]]
        O[r, c] = np.sum(patch * F)

print("Computed Output Feature Map:")
print(O)
```

Computed Output Feature Map:

```
[[4 3 4]
 [2 4 3]
 [2 3 4]]
```

Part B: End-to-End PyTorch Deep Learning Pipeline

Problem Statement: Develop a script using either TensorFlow or PyTorch that implements an end-to-end deep learning pipeline. Your solution must satisfy the following technical requirements:

1. Load raw data from a NumPy format and implement custom transformation within the pipeline.
2. Define a Convolutional Neural Network (CNN) architecture designed to process input tensors of shape `(32, 32, 3)`.
3. Incorporate at least two convolutional layers, utilizing the ReLU activation function to handle non-linearity.
4. Configure the output to classify the input into 5 distinct categories.

```
In [1]: import torch
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader
import numpy as np
```

```

# 1. Mock Raw NumPy Data Creation
num_samples = 100
# Generate random image data of shape (num_samples, channels, height, width) -> (100,
np_images = np.random.randn(num_samples, 3, 32, 32).astype(np.float32)
# Generate random labels for 5 categories (0 to 4)
np_labels = np.random.randint(0, 5, size=(num_samples,)).astype(np.int64)

# 2. Custom Dataset with Custom Transformation
class CustomDLDataset(Dataset):
    def __init__(self, images, labels, transform=None):
        self.images = images
        self.labels = labels
        self.transform = transform

    def __len__(self):
        return len(self.images)

    def __getitem__(self, idx):
        img = self.images[idx]
        label = self.labels[idx]

        # Apply custom transform if provided
        if self.transform:
            img = self.transform(img)

        return torch.tensor(img, dtype=torch.float32), torch.tensor(label, dtype=torch.int64)

# Define a custom transformation (e.g. scaling and adding a tiny bias)
def custom_transform(image_array):
    return image_array * 0.95 + 0.05

# Instantiate Dataset and DataLoader
dataset = CustomDLDataset(np_images, np_labels, transform=custom_transform)
dataloader = DataLoader(dataset, batch_size=8, shuffle=True)

# 3. CNN Architecture designed for input tensors of shape (32, 32, 3)
# PyTorch expects channels-first input: (Batch, Channels, Height, Width) -> (B, 3, 32, 32)
class SimpleCNN(nn.Module):
    def __init__(self, num_classes=5):
        super(SimpleCNN, self).__init__()

        # Conv Layer 1: Input (3, 32, 32) -> Output (16, 32, 32)
        self.conv1 = nn.Conv2d(in_channels=3, out_channels=16, kernel_size=3, padding=1)
        self.relu1 = nn.ReLU()
        # Pooling 1: (16, 32, 32) -> (16, 16, 16)
        self.pool1 = nn.MaxPool2d(kernel_size=2, stride=2)

        # Conv Layer 2: Input (16, 16, 16) -> Output (32, 16, 16)
        self.conv2 = nn.Conv2d(in_channels=16, out_channels=32, kernel_size=3, padding=1)
        self.relu2 = nn.ReLU()
        # Pooling 2: (32, 16, 16) -> (32, 8, 8)
        self.pool2 = nn.MaxPool2d(kernel_size=2, stride=2)

        # Fully Connected Layer: Input (32 * 8 * 8) -> Output (5 classes)
        self.fc = nn.Linear(32 * 8 * 8, num_classes)

```

```

    def forward(self, x):
        x = self.pool1(self.relu1(self.conv1(x)))
        x = self.pool2(self.relu2(self.conv2(x)))
        x = x.view(x.size(0), -1)  # Flatten
        logits = self.fc(x)
        return logits

# Instantiate CNN Model
model_cnn = SimpleCNN(num_classes=5)

# Verify single forward pass
for sample_images, sample_labels in dataloader:
    print(f"Input batch images shape: {sample_images.shape}")
    outputs = model_cnn(sample_images)
    print(f"Output logits shape: {outputs.shape}")
    print(f"Predictions check (argmax): {torch.argmax(outputs, dim=1).tolist()}")
    break

```

Input batch images shape: torch.Size([8, 3, 32, 32])

Output logits shape: torch.Size([8, 5])

Predictions check (argmax): [2, 2, 3, 2, 3, 0, 2, 3]